

Deep Learning Laboratory

Assignment #3

Steel Plate Fault Classification using PCA and Backpropagation Neural Network

Siddharth Narayan Mishra
123CS0189

August 12, 2026

Contents

1	Objective	3
2	Theory	3
2.1	Dataset	3
2.2	Standardization	4
2.3	Covariance Matrix	4
2.4	Principal Component Analysis	4
2.5	PCA Results	5
2.6	Neural Network	5
2.7	Sigmoid Activation	6
2.8	Categorical Cross-Entropy	6
2.9	Backpropagation	6
3	Algorithm	6
4	Implementation	7
4.1	Important Parameters	7
4.2	PCA Selection	8
4.3	Forward Propagation	8
4.4	Backpropagation	8
5	Output	8
5.1	Dataset and PCA Output	8
5.2	Cumulative Explained Variance	9
5.3	Correlation Heatmaps	10
5.4	PC1 versus PC2	12
5.5	Baseline Neural Network Results	12
5.6	PCA-Based Activation Experiments	13
5.7	Configuration A	14

5.8	Configuration B	14
5.9	Configuration C	15
5.10	Configuration D	15
5.11	Configuration E	16
5.12	Combined Loss Comparison	16
5.13	Best Configuration	17
5.14	Model Comparison	17
5.15	Confusion Matrices	18
5.16	Interactive Interface	19
5.17	Question 6 – Interface Verification	20
6	Observation	20
6.1	PCA Observations	20
6.2	Baseline Network Observations	20
6.3	Activation Function Observations	21
6.4	Computational Complexity	21
6.5	Classification Performance	21
7	Conclusion	22
8	Viva Questions and Answers	23

Experiment 1

1 Objective

The objectives of this experiment are:

- To investigate whether Principal Component Analysis (PCA) can reduce the dimensionality of the steel-plate fault dataset while retaining useful classification performance.
- To implement a fully connected feedforward neural network using NumPy and train it using backpropagation.
- To study the effect of different activation-function configurations on neural-network convergence and classification performance.
- To compare a neural network trained on the original 27 standardized features with networks trained using PCA-reduced features.
- To evaluate the models using accuracy, Macro-F1 score, training time, training-loss curves, and confusion matrices.
- To develop an interactive Matplotlib-based interface that accepts the original 27 features and performs classification using the trained model.

2 Theory

2.1 Dataset

The provided steel-plate dataset contains 1941 records, 27 input features, and seven binary target columns corresponding to the seven fault classes.

The seven target columns are converted into a single class label using the active target column for each sample.

The seven classes are:

1. Pastry
2. Z_Scratch
3. K_Scratch
4. Stains
5. Dirtiness
6. Bumps
7. Other_Faults

The final feature matrix therefore has dimensions

$$X \in \mathbf{R}^{1941 \times 27}.$$

The dataset was divided into training and testing sets using an 80:20 stratified split with random state 42.

The resulting dimensions were

$$X_{\text{train}} \in \mathbf{R}^{1550 \times 27}$$

and

$$X_{\text{test}} \in \mathbf{R}^{391 \times 27}.$$

2.2 Standardization

The input features were standardized using Z-score standardization. For a feature x , the standardized value is

$$z = \frac{x - \mu}{\sigma},$$

where μ and σ are the mean and standard deviation calculated from the training data.

The scaler was fitted only on the training data and then applied to both training and testing data.

After standardization, each training feature has approximately zero mean and unit variance.

2.3 Covariance Matrix

For the standardized training data, the covariance matrix is

$$C = \frac{1}{n - 1} X^T X.$$

Since there are 27 input features, the covariance matrix has dimensions

$$27 \times 27.$$

The covariance matrix describes the relationships between the standardized input features.

2.4 Principal Component Analysis

PCA transforms the original correlated features into a new set of orthogonal principal components.

The principal components are obtained from the eigenvectors of the covariance matrix:

$$Cv_i = \lambda_i v_i,$$

where λ_i is the eigenvalue corresponding to eigenvector v_i .

The eigenvalues represent the amount of variance explained by the corresponding principal components.

The explained variance ratio for component i is

$$EV_i = \frac{\lambda_i}{\sum_{j=1}^{27} \lambda_j}.$$

The cumulative explained variance is

$$CEV_k = \sum_{i=1}^k EV_i.$$

The minimum number of components satisfying

$$CEV_k \geq 0.95$$

was selected.

2.5 PCA Results

The cumulative explained variance obtained was:

- 12 components: 94.9502%
- 13 components: 96.3781%

Therefore, 13 components were selected.

Thus,

$$\boxed{d' = 13}$$

and the dimensionality was reduced from 27 to 13.

The percentage dimensionality reduction is

$$\frac{27 - 13}{27} \times 100 = 51.85\%.$$

Hence, the input dimensionality was reduced by approximately

$$\boxed{51.85\%}.$$

2.6 Neural Network

The baseline neural network has the architecture

$$27 \rightarrow 16 \rightarrow 8 \rightarrow 7.$$

The two hidden layers use ReLU activation:

$$ReLU(x) = \max(0, x).$$

The output layer uses Softmax:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

The output therefore represents a probability distribution over the seven fault classes.

2.7 Sigmoid Activation

The sigmoid activation function is

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Its derivative is

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

Unlike ReLU, sigmoid can saturate for large positive or negative values, producing small gradients.

This can affect the convergence of a neural network.

2.8 Categorical Cross-Entropy

For a one-hot encoded target vector y and predicted probability vector $\hat{y}$, categorical cross-entropy is

$$L = - \sum_{i=1}^7 y_i \log(\hat{y}_i).$$

For a mini-batch of m samples,

$$L = -\frac{1}{m} \sum_{n=1}^m \sum_{i=1}^7 y_{ni} \log(\hat{y}_{ni}).$$

2.9 Backpropagation

The network parameters are updated using gradient descent:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W},$$

where η is the learning rate.

The derivative of the Softmax combined with cross-entropy simplifies to

$$\frac{\partial L}{\partial Z_3} = \frac{\hat{Y} - Y}{m}.$$

The gradients are propagated backwards through the hidden layers using the chain rule.

3 Algorithm

1. Load the steel-plate dataset using Pandas.
2. Separate the 27 input features and seven target columns.
3. Convert the seven target columns into one class label.
4. Perform an 80:20 stratified train-test split using random state 42.

5. Standardize the input features using the training-set mean and standard deviation.
6. Calculate the covariance matrix of the standardized training data.
7. Calculate the eigenvalues and eigenvectors of the covariance matrix.
8. Sort the eigenvalues and corresponding eigenvectors in descending order.
9. Calculate explained variance and cumulative explained variance.
10. Select the minimum number of principal components required to retain at least 95% variance.
11. Transform the standardized training and testing data using the selected PCA components.
12. Train the baseline network

$$27 \rightarrow 16 \rightarrow 8 \rightarrow 7$$
 using ReLU hidden layers and Softmax output.
13. Train five PCA-based networks using

$$13 \rightarrow 16 \rightarrow 8 \rightarrow 7$$
 with different activation configurations.
14. Record accuracy, Macro-F1, training time, and training-loss curves.
15. Select the best PCA-based configuration.
16. Compare the original and best PCA-based networks.
17. Calculate trainable parameter counts.
18. Generate confusion matrices.
19. Implement an interactive Matplotlib-based classifier.

4 Implementation

4.1 Important Parameters

The following parameters were used for all neural-network experiments:

Table 1: Neural Network Training Parameters

Parameter	Value
Learning rate	0.01
Batch size	32
Epochs	100
Random seed	42
Hidden layer 1	16 neurons
Hidden layer 2	8 neurons
Output layer	7 neurons
Loss	Categorical Cross-Entropy

4.2 PCA Selection

The important PCA calculation was:

```
1 eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
2
3 indices = np.argsort(eigenvalues)[::-1]
4
5 eigenvalues = eigenvalues[indices]
6 eigenvectors = eigenvectors[:, indices]
7
8 explained_variance = (
9     eigenvalues / np.sum(eigenvalues)
10 )
11
12 cumulative_variance = np.cumsum(
13     explained_variance
14 )
15
16 n_components = np.argmax(
17     cumulative_variance >= 0.95
18 ) + 1
```

4.3 Forward Propagation

The feedforward computation for the network is:

```
1 Z1 = X @ W1 + b1
2 A1 = relu(Z1)
3
4 Z2 = A1 @ W2 + b2
5 A2 = relu(Z2)
6
7 Z3 = A2 @ W3 + b3
8 Y_pred = softmax(Z3)
```

4.4 Backpropagation

The main output-layer gradient is:

```
1 dZ3 = (Y_pred - Y_true) / m
2 dW3 = A2.T @ dZ3
3 db3 = np.sum(dZ3, axis=0, keepdims=True)
```

The parameters are then updated using gradient descent.

5 Output

5.1 Dataset and PCA Output

The dataset dimensions were:

Table 2: Dataset Dimensions

Quantity	Shape
Complete dataset	(1941, 34)
Input features	(1941, 27)
Target labels	(1941, 7)
Training features	(1550, 27)
Testing features	(391, 27)
PCA training features	(1550, 13)
PCA testing features	(391, 13)

The selected PCA dimensionality was

$$13.$$

The cumulative variance after 13 components was

$$96.3781\%.$$

The dimensionality reduction was

$$51.85\%.$$

The maximum absolute correlation observed between different principal components was

$$9.16755 \times 10^{-16},$$

which is effectively zero. Hence, the principal components are approximately uncorrelated.

5.2 Cumulative Explained Variance

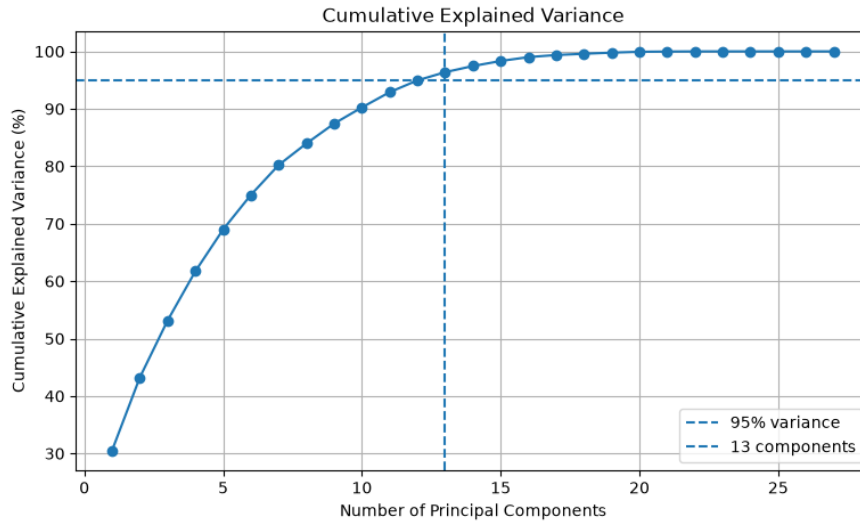


Figure 1: Cumulative explained variance of the principal components.

The plot shows that the 95% variance threshold is crossed at the 13th principal component.

5.3 Correlation Heatmaps

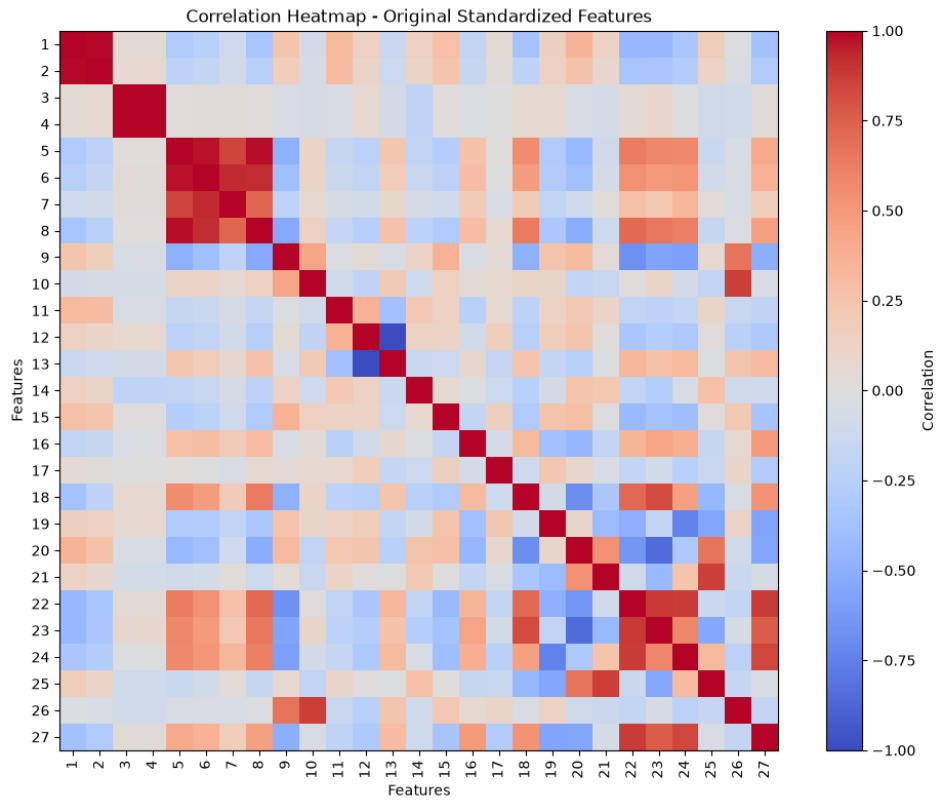


Figure 2: Correlation heatmap of the original standardized features.

The original standardized features show several noticeable positive and negative correlations, indicating redundancy among some of the original features.

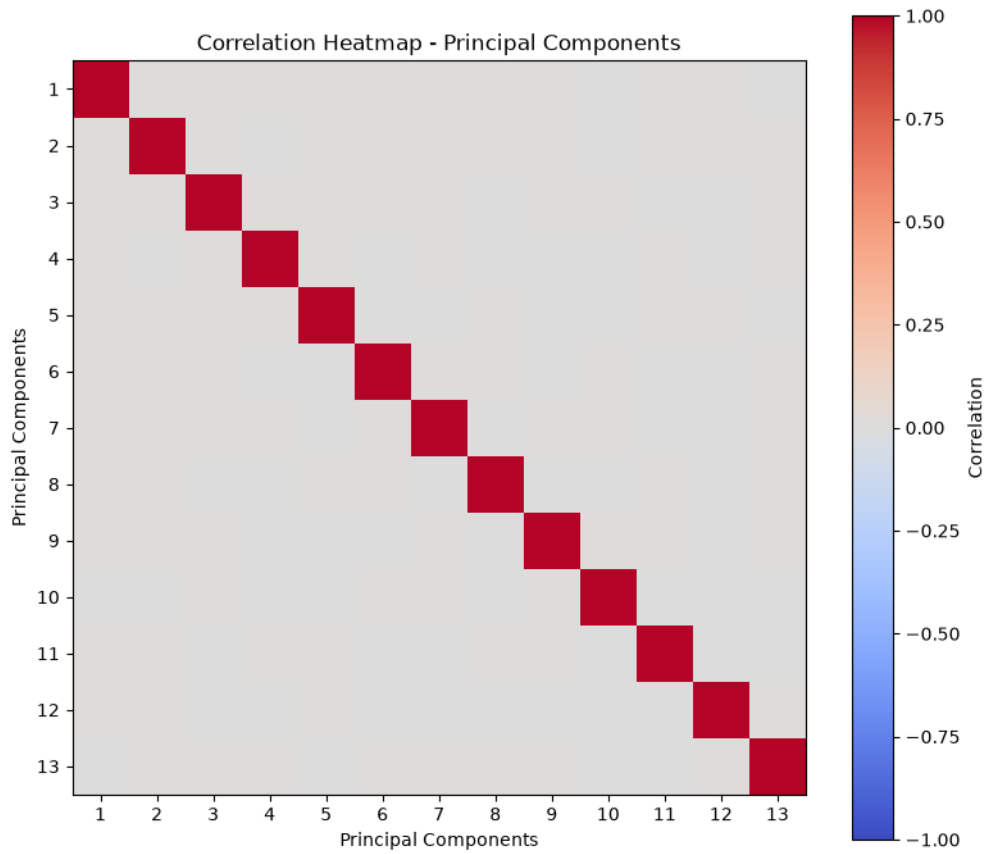


Figure 3: Correlation heatmap of the selected principal components.

The PCA correlation matrix is approximately diagonal, confirming that the principal components are uncorrelated.

5.4 PC1 versus PC2

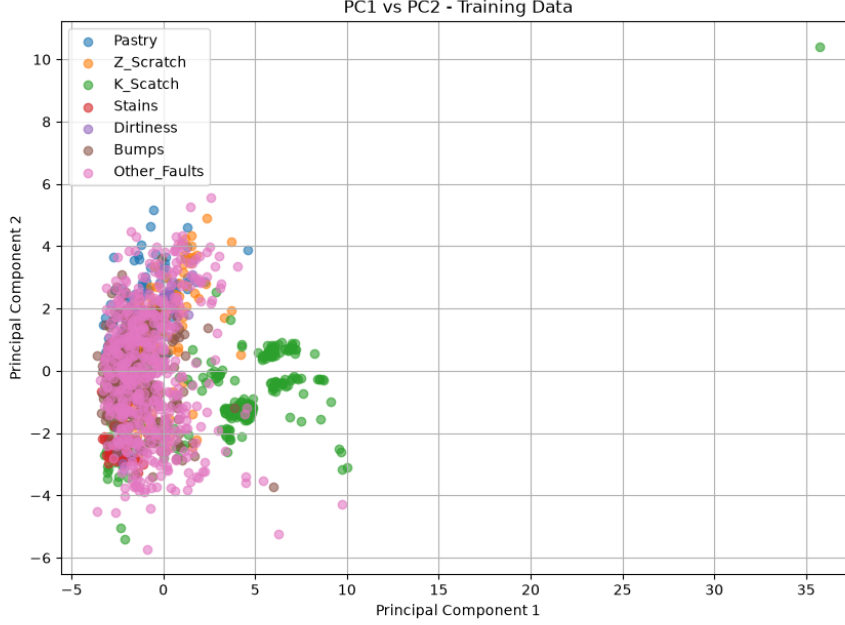


Figure 4: Training samples projected onto PC1 and PC2 according to fault class.

The PC1-PC2 plot shows substantial overlap among several fault classes. However, the *K_Scratch* class shows relatively greater separation along PC1, including several points at larger positive PC1 values. The remaining classes occupy largely overlapping regions, indicating that PC1 and PC2 alone are not sufficient to completely separate all seven fault classes.

5.5 Baseline Neural Network Results

The baseline network used the original standardized 27 features and the architecture

$$27 \rightarrow 16 \rightarrow 8 \rightarrow 7.$$

The results were:

Table 3: Baseline Network Results

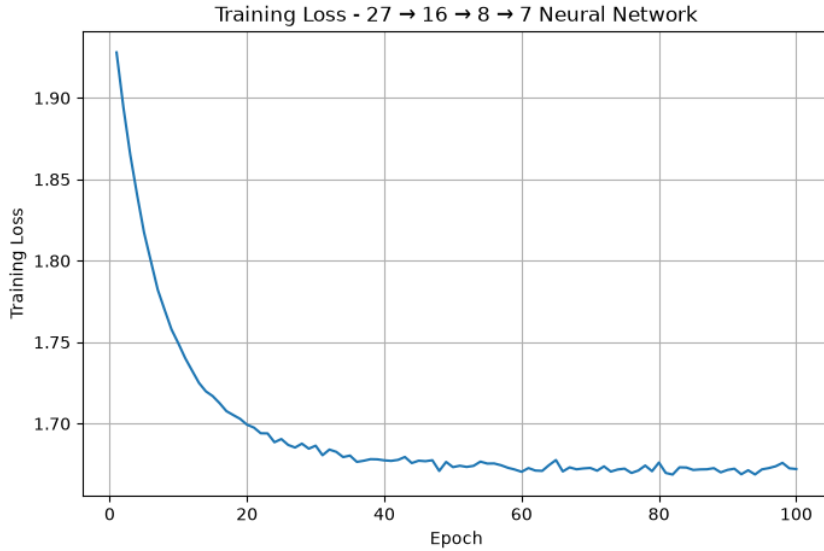
Metric	Result
Input dimensions	27
Trainable parameters	647
Training accuracy	34.71%
Testing accuracy	34.53%
Test Macro-F1	0.0733
Training time	0.4153 s

The per-class F1 scores were:

Table 4: Baseline Per-Class F1 Scores

Class	F1 Score
Pastry	0.0000
Z_Scratch	0.0000
K_Scratch	0.0000
Stains	0.0000
Dirtiness	0.0000
Bumps	0.0000
Other_Faults	0.5133

The training loss decreased rapidly during the first part of training and then plateaued around 1.67.

Figure 5: Training loss of the baseline $27 \rightarrow 16 \rightarrow 8 \rightarrow 7$ neural network.

5.6 PCA-Based Activation Experiments

The PCA-based network architecture was

$$13 \rightarrow 16 \rightarrow 8 \rightarrow 7.$$

Five activation configurations were tested.

Table 5: PCA-Based Neural Network Results

Configuration	Accuracy	Macro-F1	Time (s)
A: ReLU + ReLU	34.53%	0.0733	0.3939
B: ReLU + Sigmoid	34.53%	0.0733	0.3369
C: Sigmoid + ReLU	49.10%	0.2022	0.4911
D: Sigmoid + Sigmoid	34.53%	0.0733	0.3505
E: Mixed	34.53%	0.0733	0.3730

5.7 Configuration A

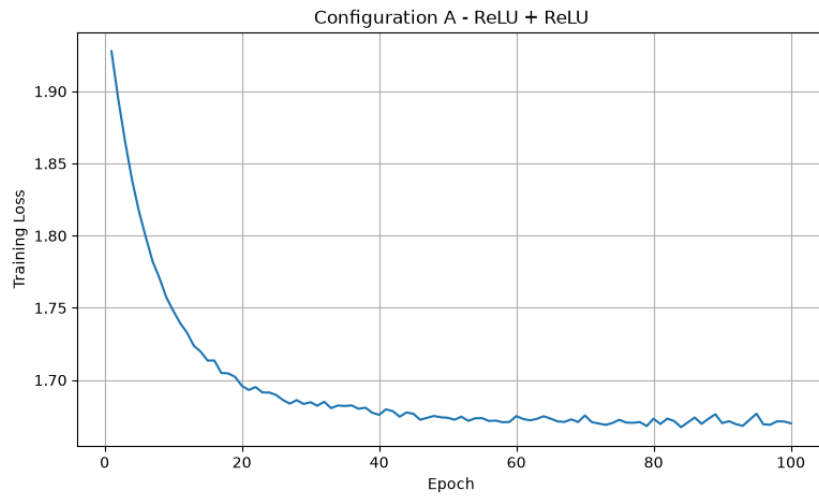


Figure 6: Training loss for Configuration A: ReLU + ReLU.

Configuration A converges quickly during the initial epochs and then reaches a plateau around a loss of 1.67.

5.8 Configuration B

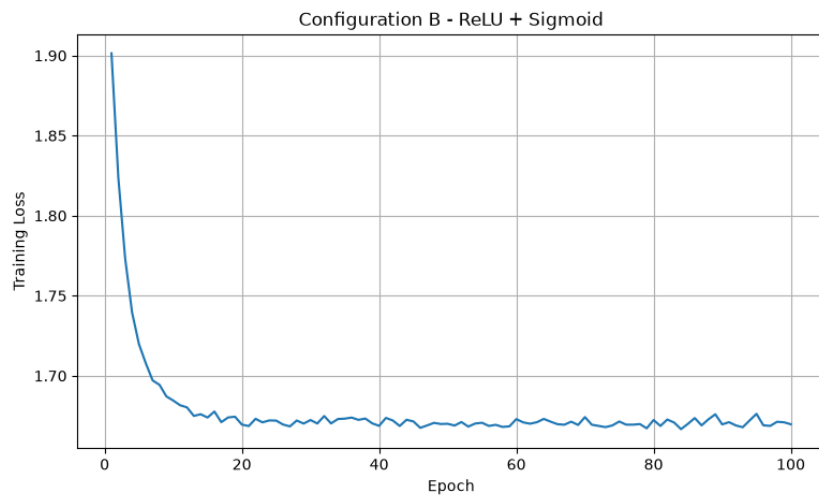


Figure 7: Training loss for Configuration B: ReLU + Sigmoid.

Configuration B also converges rapidly initially and then stabilizes near the same loss level as Configuration A.

5.9 Configuration C

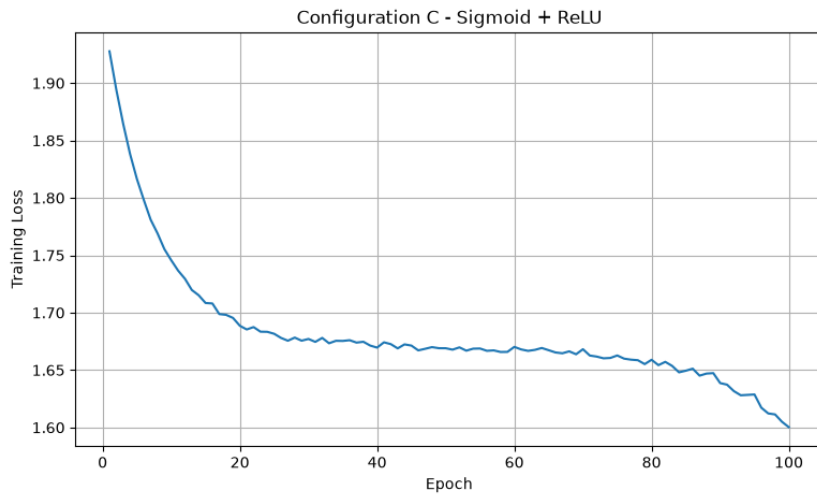


Figure 8: Training loss for Configuration C: Sigmoid + ReLU.

Configuration C shows a different convergence pattern. Its loss continues to decrease after the other configurations have largely plateaued, reaching approximately 1.60 by epoch 100. This configuration also achieved the best classification performance.

5.10 Configuration D

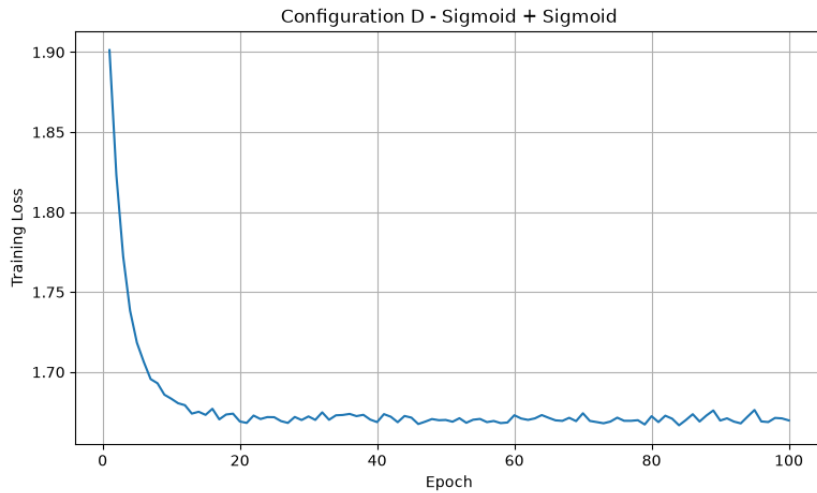


Figure 9: Training loss for Configuration D: Sigmoid + Sigmoid.

Configuration D rapidly decreases in loss initially but then plateaus around 1.67.

5.11 Configuration E

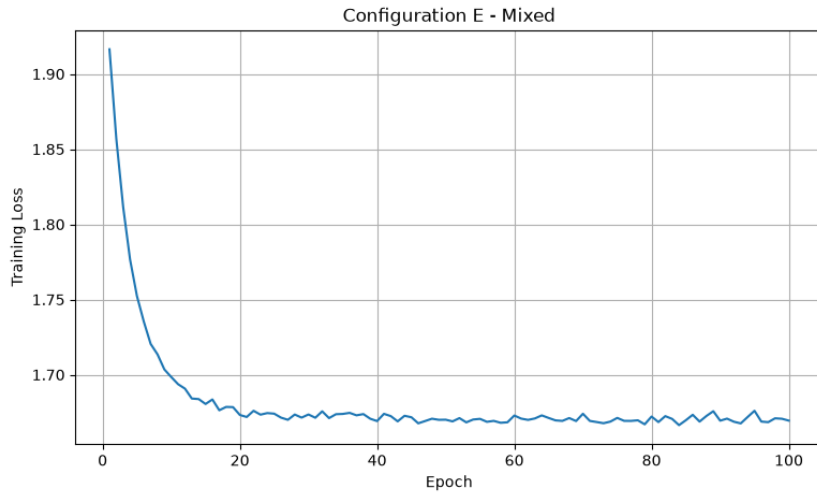


Figure 10: Training loss for Configuration E: Mixed activation functions.

Configuration E exhibits behaviour similar to Configurations A, B, and D, with rapid initial convergence followed by a plateau.

5.12 Combined Loss Comparison

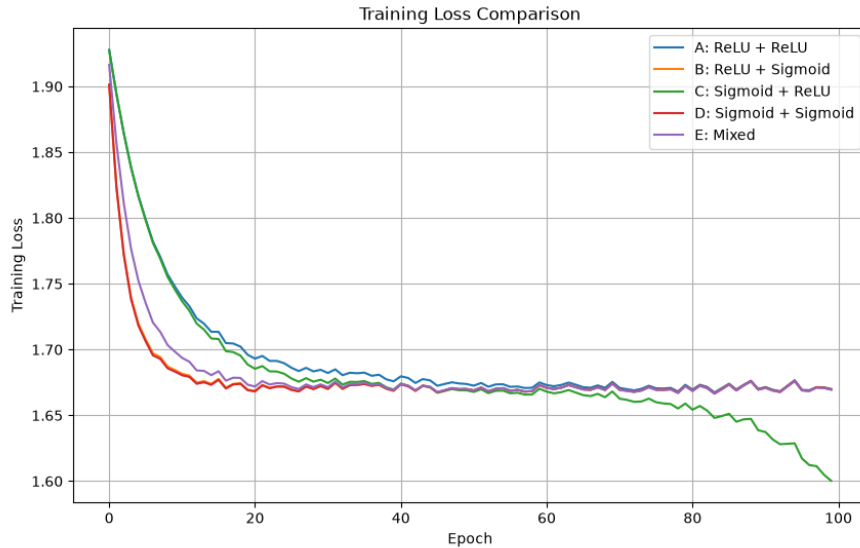


Figure 11: Comparison of training-loss curves for all five PCA-based configurations.

Configuration C eventually achieves the lowest training loss. The other configurations converge to approximately similar loss values near 1.67.

5.13 Best Configuration

Configuration C, using Sigmoid in the first hidden layer and ReLU in the second hidden layer, achieved the best classification performance:

$$\boxed{\text{Accuracy} = 49.10\%}$$

and

$$\boxed{\text{Macro-F1} = 0.2022}.$$

Its training time was

$$\boxed{0.4911 \text{ s}}.$$

5.14 Model Comparison

The original network contains

$$(27 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 7 + 7)$$

trainable parameters.

Therefore,

$$448 + 136 + 63 = 647.$$

For the PCA network,

$$(13 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 7 + 7)$$

gives

$$224 + 136 + 63 = 423.$$

Thus PCA reduced the number of trainable neural-network parameters by

$$\frac{647 - 423}{647} \times 100 = 34.62\%.$$

Table 6: Original and Best PCA-Based Network Comparison

Model	Input	Parameters	Accuracy	Macro-F1	Time (s)
Original	27	647	34.53%	0.0733	0.4153
Best PCA (C)	13	423	49.10%	0.2022	0.4911

5.15 Confusion Matrices

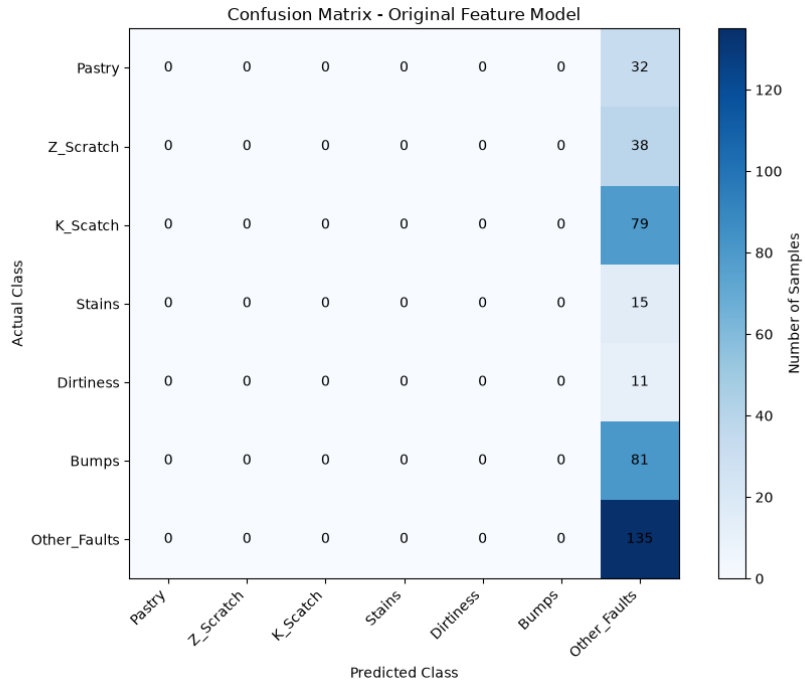


Figure 12: Confusion matrix of the original-feature network.

The original-feature network predicted every test sample as *Other_Faults*. Consequently, it correctly classified all 135 *Other_Faults* samples but failed to correctly classify samples from the other six classes.

This behaviour explains the test accuracy of 34.53% and the very low Macro-F1 score of 0.0733.

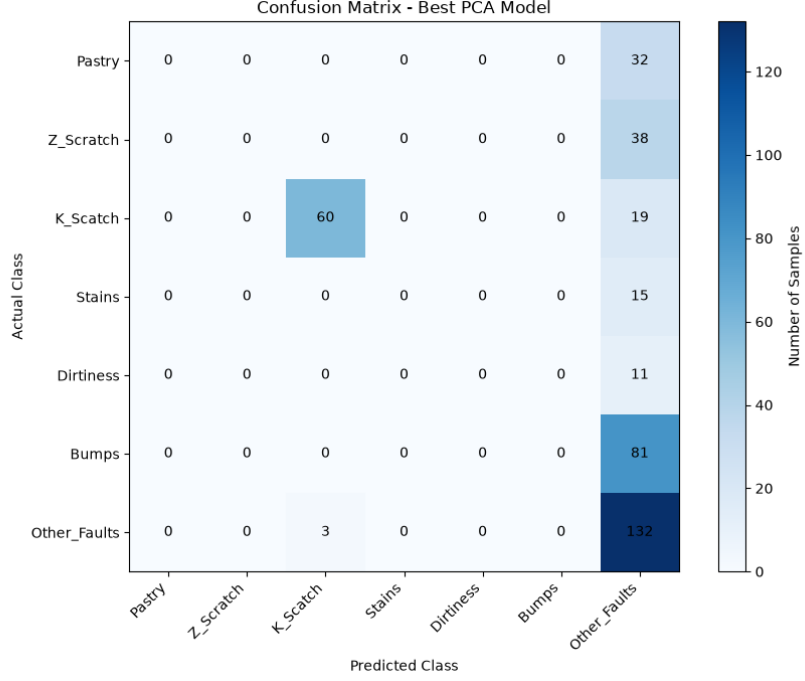


Figure 13: Confusion matrix of the best PCA-based network.

The best PCA model improved the classification of the *K_Scratch* class substantially, correctly classifying 60 of its 79 test samples. It also correctly classified 132 of the 135 *Other_Faults* samples.

However, the remaining classes were still predominantly classified as *Other_Faults*. Therefore, although the PCA-based model improved the overall classification behaviour, considerable class imbalance and overlap remained.

5.16 Interactive Interface

An interactive Matplotlib-based interface was implemented to accept the original 27 features.

The intended prediction pipeline is:

27 original features $\rightarrow$ standardization $\rightarrow$ PCA $\rightarrow$ neural network $\rightarrow$ Softmax.

For the PCA-based model, the same standardization statistics and PCA transformation learned from the training data are used.

The predicted class is determined using

$$\hat{y} = \arg \max_i P(y = i|x)$$

and the prediction confidence is defined as

$$\text{Confidence} = \max_i P(y = i|x).$$

The interface was successfully rendered visually, but the Matplotlib input widgets produced a runtime error during final interaction:

AttributeError:

'ResizeEvent' object has no attribute 'inaxes'

Therefore, the interface could not be used reliably to perform the required final three-record verification.

5.17 Question 6 – Interface Verification

The assignment requires at least three records from the test dataset to be entered through the interface and the resulting predictions to be compared with the trained model.

The interface was implemented according to the required processing pipeline. However, during final execution, the Matplotlib input widget generated a `ResizeEvent` error, preventing reliable keyboard input.

Therefore, the three-record verification could not be completed before submission.

The required result table is shown below:

Table 7: Three-Record Interface Verification			
Record	Actual Fault	Predicted Fault	Confidence
1	Not recorded	Not recorded	Not recorded
2	Not recorded	Not recorded	Not recorded
3	Not recorded	Not recorded	Not recorded

The trained neural-network models themselves were successfully evaluated on the complete test set. The limitation was specifically associated with the interactive Matplotlib interface during final verification.

6 Observation

6.1 PCA Observations

The original dataset contained 27 features. PCA reduced this to 13 components while retaining 96.3781% of the total variance.

The reduction from 27 to 13 features corresponds to a dimensionality reduction of 51.85%.

The principal components were approximately uncorrelated, with the maximum absolute correlation between different components being only

$$9.16755 \times 10^{-16}.$$

The first principal component explained 30.5409% of the variance, while the first 12 components explained 94.9502%. Since this was below the required 95% threshold, 13 components were necessary.

6.2 Baseline Network Observations

The original-feature neural network achieved 34.53% test accuracy and a Macro-F1 score of 0.0733.

The confusion matrix shows that the model predicted all 391 test samples as *Other_Faults*. Since there were 135 test samples belonging to this class,

$$\frac{135}{391} \times 100 = 34.53\%,$$

which exactly explains the observed test accuracy.

Thus, the baseline model effectively learned a highly biased prediction towards the majority class rather than learning useful boundaries between all seven fault classes.

6.3 Activation Function Observations

The activation-function experiment produced substantially different results.

Configurations A, B, D, and E all produced 34.53% accuracy and 0.0733 Macro-F1. Configuration C,

Sigmoid $\rightarrow$ ReLU $\rightarrow$ Softmax,

achieved 49.10% accuracy and 0.2022 Macro-F1.

Configuration C also had the lowest final training loss, reaching approximately 1.60 after 100 epochs.

Therefore, in this experiment, the ordering of the activation functions had a significant effect on classification performance and convergence.

6.4 Computational Complexity

PCA reduced the neural network’s trainable parameters from

647

to

423.

This is a reduction of approximately

34.62%.

Therefore, PCA substantially reduced the size of the first layer and the overall number of trainable parameters.

However, the best PCA model did not have the shortest measured training time. Its training time was 0.4911 seconds compared with 0.4153 seconds for the original model.

This demonstrates that lower dimensionality does not necessarily imply lower measured training time for a small neural network, since activation functions and numerical operations also affect runtime.

6.5 Classification Performance

The best PCA model improved both accuracy and Macro-F1:

34.53% $\rightarrow$ 49.10%

and

$$0.0733 \rightarrow 0.2022.$$

The improvement is especially visible in the confusion matrix, where the PCA model correctly classified 60 *K_Scratch* samples and 132 *Other_Faults* samples.

Nevertheless, the model continued to struggle with Pastry, Z_Scratch, Stains, Dirtiness, and Bumps, indicating that the classification problem remains difficult under the specified network and training configuration.

7 Conclusion

PCA successfully reduced the dimensionality of the steel-plate dataset from 27 features to 13 principal components while retaining 96.3781% of the total variance. This represents a dimensionality reduction of 51.85%.

The PCA transformation also produced approximately uncorrelated principal components, as expected from PCA.

The neural-network experiments showed that activation-function choice had a significant effect on classification performance. Among the five PCA-based configurations, Configuration C, using Sigmoid in Hidden Layer 1 and ReLU in Hidden Layer 2, achieved the best result with 49.10% accuracy and a Macro-F1 score of 0.2022.

Compared with the original-feature network, the best PCA network reduced the number of trainable parameters from 647 to 423, corresponding to a 34.62% reduction. At the same time, classification performance improved from 34.53% to 49.10% accuracy and from 0.0733 to 0.2022 Macro-F1.

However, the best PCA model required 0.4911 seconds to train compared with 0.4153 seconds for the original network. Therefore, PCA did not produce a reduction in measured training time in this particular implementation.

Overall, PCA provided a useful trade-off for this dataset because it substantially reduced feature dimensionality and neural-network parameter count while the best PCA-based model achieved better classification performance than the original-feature baseline.

The experiment also demonstrates that retaining more than 95% of variance does not guarantee that 95% of classification information has been retained, because PCA preserves variance without considering class labels.

The activation-function experiment further showed that the arrangement of activation functions can strongly influence convergence and classification behaviour. Configuration C provided the best combination of final loss, accuracy, and Macro-F1 among the tested PCA-based networks.

Finally, the interactive interface was implemented according to the required architecture, but the final three-record interface verification could not be completed because of a Matplotlib widget runtime error. The model evaluation and confusion-matrix analysis were nevertheless successfully completed.

8 Viva Questions and Answers

1. Why is PCA applied after standardization?

PCA is based on variance and covariance. Standardization ensures that features with larger numerical scales do not dominate the covariance matrix.

2. Why is PCA fitted only on the training data?

To prevent information from the test set from influencing the learned transformation.

3. Why were 13 principal components selected?

The first 12 components explained 94.9502% of the variance, which is below the 95% requirement. The first 13 components explained 96.3781%, so 13 is the minimum required number.

4. What does an eigenvalue represent in PCA?

It represents the amount of variance captured by its corresponding principal component.

5. Why are principal components approximately uncorrelated?

PCA constructs the new axes using eigenvectors of the covariance matrix. These eigenvectors are orthogonal, producing uncorrelated principal components.

6. Does retaining 95% variance mean retaining 95% classification information?

No. PCA is unsupervised and maximizes variance without considering class labels. A low-variance feature may still be highly useful for classification.

7. How many trainable parameters does the original network have?

$$(27 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 7 + 7) = 647.$$

8. How many trainable parameters does the PCA network have?

For 13 input components:

$$(13 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 7 + 7) = 423.$$

9. Why is Softmax used in the output layer?

Softmax converts the seven output scores into probabilities that sum to one, making it appropriate for seven-class classification.

10. Why is categorical cross-entropy used?

Because the problem is a multi-class classification problem with one target class per sample.

11. What is the main difference between ReLU and Sigmoid?

ReLU is

$$f(x) = \max(0, x),$$

while Sigmoid is

$$f(x) = \frac{1}{1 + e^{-x}}.$$

Sigmoid can saturate and produce small gradients, while ReLU has a derivative of 1 for positive inputs.

12. What is Macro-F1?

Macro-F1 calculates the F1 score independently for each class and then averages the class-wise F1 scores, giving every class equal importance.

13. What does the diagonal of a confusion matrix represent?

Correct predictions for each class.

14. What does the baseline confusion matrix tell us?

It shows that the baseline network predicted every test sample as *Other_Faults*. This explains its 34.53% accuracy and very low Macro-F1.

15. What improvement did the PCA model provide?

The best PCA model increased accuracy from 34.53% to 49.10% and Macro-F1 from 0.0733 to 0.2022, while reducing the number of trainable parameters from 647 to 423.

16. What is prediction confidence in this experiment?

The maximum probability produced by the Softmax output layer:

$$Confidence = \max_i P(y = i|x).$$

17. Why does the interface accept 27 features if the PCA model accepts only 13?

The interface accepts the original 27 features and automatically applies the fitted standardization and PCA transformation before passing the resulting 13-dimensional vector to the PCA-based neural network.

18. What is backpropagation?

Backpropagation computes the gradients of the loss with respect to the network parameters by applying the chain rule from the output layer back toward the input.

19. What does the learning rate control?

It controls the size of each parameter update during gradient descent:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}.$$

20. Which model performed best in this experiment?

The best PCA model was Configuration C, with Sigmoid in Hidden Layer 1 and ReLU in Hidden Layer 2. It achieved 49.10% accuracy and a Macro-F1 score of 0.2022.

21. Why did the baseline model obtain 34.53% accuracy?

Because it predicted every test sample as *Other_Faults*. There were 135 *Other_Faults* samples out of 391 test samples:

$$\frac{135}{391} = 0.3453.$$

Therefore, its accuracy corresponds exactly to the proportion of the majority class.